Министерство науки и высшего образования РФ

Пензенский государственный университет

Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №8

по курсу «Логика и основы алгоритмизации в инженерных задачах»

на тему «Обход графа в ширину»

Выполнили студенты группы 25ВВВ2

Лампетов А.С.

Ковтун А.А.

Приняли:

к.т.н., доцент Юрова О.В.

к.т.н., доцент Деев М.В.

Пенза 2026

Тема: Обход графа в ширину

Цель работы

Изучение алгоритма обхода графа в ширину и реализация на языке Go его варианта для матрицы смежности с очередью на основе среза и с самостоятельно реализованной очередью на основе односвязного списка, а также варианта для списков смежности; сравнение времени работы реализаций и оценка сложности обхода.

Лабораторное задание

1. Сгенерировать с помощью генератора случайных чисел матрицу смежности неориентированного графа G и вывести её на экран.
2. Для сгенерированного графа выполнить обход в ширину, реализованный в соответствии с приведённым в методических указаниях описанием; в качестве очереди использовать срез.
3. Реализовать процедуру обхода в ширину для графа, представленного списками смежности.
4. Для матричной формы представления графа реализовать алгоритм обхода в ширину с использованием очереди, построенной на основе односвязного списка, самостоятельно созданного в лабораторной работе №3.
5. Оценить время работы двух реализаций обхода в ширину (со срезом и с самостоятельно созданной очередью) для графов разных порядков.

Ход работы

Граф хранится в структуре Graph, которая содержит число вершин N и матрицу смежности Mat. Функция NewGraph заполняет верхний треугольник матрицы случайными значениями: ребро между вершинами i и j создаётся с вероятностью p процентов, после чего значение копируется в симметричную ячейку, так что граф получается неориентированным. Кроме того, каждая диагональная ячейка с той же вероятностью получает единицу, то есть в графе

могут появляться петли; на результат обхода они не влияют, так как петля ведёт в уже помеченную вершину. Вершины нумеруются с единицы, при вводе с клавиатуры номер приводится к индексу массива вычитанием единицы.

```
type Graph struct {
    N   int
    Mat [][]int
}

func NewGraph(n int, p float64) Graph {
    g := Graph{
        N:   n,
        Mat: make([][]int, n),
    }

    for i := range g.Mat {
        g.Mat[i] = make([]int, n)
    }

    for i := 0; i < n; i++ {
        for j := i + 1; j < n; j++ {
            if rand.Float64()*100 < p {
                g.Mat[i][j] = 1
                g.Mat[j][i] = 1
            }
        }
    }

    for i := 0; i < n; i++ {
        if rand.Float64()*100 < p {
            g.Mat[i][i] = 1
        }
    }

    return g
}
```

Программа управляется текстовым меню: создание случайного графа, вывод матрицы и списков смежности, четыре варианта обхода в ширину (по матрице со срезом, по матрице с самостоятельной очередью, по спискам смежности со срезом и по спискам смежности с самостоятельной очередью), сравнение времени двух реализаций, а также указание вероятности появления ребра. Вводимые значения проверяются: при некорректном вводе запрос повторяется. Если граф ещё не создан, обходы не выполняются, и пользователю выводится соответствующее сообщение.

Очередь на основе односвязного списка

Для второго варианта обхода использована очередь, построенная на основе односвязного списка, как в лабораторной работе №3. Узел Node хранит значение Value и указатель Next на следующий узел. Структура LinkedQueue хранит указатели на первый (Head) и последний (Last) узлы, благодаря чему и добавление в конец, и извлечение из начала выполняются за время O(1). Метод Push создаёт новый узел: если очередь пуста (Last равен nil), он становится одновременно первым и последним, иначе присоединяется после текущего последнего узла. Метод Pop возвращает значение первого узла и признак успеха, для пустой очереди возвращается false; если после извлечения очередь опустела, указатель Last обнуляется. Метод IsEmpty проверяет, что указатель Head равен nil.

```
type Node struct {
    Value int
    Next  *Node
}

type LinkedQueue struct {
    Head *Node
    Last *Node
}

func (q *LinkedQueue) Push(value int) {
    node := &Node{
        Value: value,
        Next:  nil,
    }

    if q.Last == nil {
        q.Head = node
        q.Last = node
        return
    }

    q.Last.Next = node
    q.Last = node
}

func (q *LinkedQueue) Pop() (int, bool) {
    if q.Head == nil {
        return 0, false
    }

    value := q.Head.Value
    q.Head = q.Head.Next

    if q.Head == nil {
        q.Last = nil
    }
```

```
    return value, true
}

func (q *LinkedQueue) IsEmpty() bool {
    return q.Head == nil
}
```

Обход в ширину по матрице смежности

Алгоритм реализован в методе BFSMatrix, который объединяет обе части алгоритма из методических указаний. Подготовительная часть (пп. 1.1–1.3): создаётся массив отметок visited, в котором все вершины сначала не посещены, и подготавливается срез order, в который записываются номера вершин в порядке обхода. Внешний цикл сначала берёт начальную вершину, выбранную пользователем, а затем по очереди все остальные; если вершина ещё не посещена, из неё запускается обход, поэтому проходятся все компоненты связности. Процедура BFS(v) (пп. 2.1–2.12) выполнена внутри этого цикла: создаётся пустая очередь, в неё помещается вершина v, и она помечается посещённой. Пока очередь не пуста, из неё извлекается первый элемент (queue = queue[1:]), его номер записывается в order, после чего просматривается соответствующая строка матрицы, и каждая смежная непосещённая вершина добавляется в конец очереди и сразу помечается. Пометка при добавлении, а не при извлечении гарантирует, что вершина попадёт в очередь не более одного раза. В качестве очереди используется срез целых чисел, так как в стандартной библиотеке Go нет готового класса очереди.

```
func (g Graph) BFSMatrix(start int, order *[]int) {
    visited := make([]bool, g.N)
    *order = make([]int, 0, g.N)

    for startVertex := 0; startVertex < g.N; startVertex++ {
        v := start
        if startVertex > 0 {
            v = startVertex - 1
            if v >= start {
                v++
            }
        }

        if visited[v] {
            continue
        }

        queue := make([]int, 0, g.N)
```

```
        queue = append(queue, v)
        visited[v] = true

        for len(queue) > 0 {
            current := queue[0]
            queue = queue[1:]

            *order = append(*order, current)

            for to := 0; to < g.N; to++ {
                if g.Mat[current][to] == 1 && !visited[to] {
                    queue = append(queue, to)
                    visited[to] = true
                }
            }
        }
    }
}
```

Обход с самостоятельно реализованной очередью

Метод BFSLinkedQueue повторяет логику BFSMatrix, но вместо среза использует очередь LinkedQueue: вызов append заменён методом Push, извлечение первого элемента выполняется методом Pop, а условием продолжения цикла служит !queue.IsEmpty(). Новая очередь создаётся для каждой компоненты связности. Порядок обхода получается таким же, как у предыдущей реализации.

```
func (g Graph) BFSLinkedQueue(start int, order *[]int) {
    visited := make([]bool, g.N)
    *order = make([]int, 0, g.N)

    for startVertex := 0; startVertex < g.N; startVertex++ {
        v := start
        if startVertex > 0 {
            v = startVertex - 1
            if v >= start {
                v++
            }
        }

        if visited[v] {
            continue
        }

        var queue LinkedQueue

        queue.Push(v)
        visited[v] = true

        for !queue.IsEmpty() {
            current, _ := queue.Pop()
            *order = append(*order, current)

            for to := 0; to < g.N; to++ {
                if g.Mat[current][to] == 1 && !visited[to] {
```

```
                    queue.Push(to)
                    visited[to] = true
                }
            }
        }
    }
}
```

Обход по спискам смежности

Метод AdjacencyList преобразует матрицу в списки смежности: для каждой строки матрицы номера столбцов с единицами добавляются в список соответствующей вершины. Метод BFSAdjacencyList выполняет тот же обход, но вместо просмотра всей строки матрицы перебирает только список соседей текущей вершины; очередью служит срез. Метод BFSAdjacencyListQ отличается тем, что использует очередь LinkedQueue, и возвращает порядок обхода в виде среза, а не записывает его по указателю.

```
func (g Graph) AdjacencyList() [][]int {
    list := make([][]int, g.N)

    for i := 0; i < g.N; i++ {
        for j := 0; j < g.N; j++ {
            if g.Mat[i][j] == 1 {
                list[i] = append(list[i], j)
            }
        }
    }

    return list
}

func (g Graph) BFSAdjacencyList(start int, order *[]int) {
    list := g.AdjacencyList()

    visited := make([]bool, g.N)
    *order = make([]int, 0, g.N)

    for startVertex := 0; startVertex < g.N; startVertex++ {
        v := start
        if startVertex > 0 {
            v = startVertex - 1
            if v >= start {
                v++
            }
        }

        if visited[v] {
            continue
        }

        queue := make([]int, 0, g.N)

        queue = append(queue, v)
        visited[v] = true
```

```
        for len(queue) > 0 {
            current := queue[0]
            queue = queue[1:]

            *order = append(*order, current)

            for _, to := range list[current] {
                if !visited[to] {
                    queue = append(queue, to)
                    visited[to] = true
                }
            }
        }
    }
}

func (g Graph) BFSAdjacencyListQ(start int) []int {
    list := g.AdjacencyList()

    visited := make([]bool, g.N)
    order := make([]int, 0, g.N)

    for startVertex := 0; startVertex < g.N; startVertex++ {
        v := start
        if startVertex > 0 {
            v = startVertex - 1
            if v >= start {
                v++
            }
        }

        if visited[v] {
            continue
        }

        var queue LinkedQueue

        queue.Push(v)
        visited[v] = true

        for !queue.IsEmpty() {
            current, _ := queue.Pop()

            order = append(order, current)

            for _, to := range list[current] {
                if !visited[to] {
                    queue.Push(to)
                    visited[to] = true
                }
            }
        }
    }

    return order
}
```

Сравнение времени работы

Для сравнения двух реализаций обхода по матрице смежности используется функция compareTime. Она запускает в отдельных горутинах

методы BFSMatrix и BFSLinkedQueue, передавая их в функцию checkTime как значения. Функция checkTime измеряет время выполнения с помощью time.Now и time.Since и записывает его в переменную, переданную по указателю; завершение горутин ожидается с помощью sync.WaitGroup. После вызова wg.Wait функция выводит время обеих реализаций и, если параметр writeResult равен true, порядок обхода. Поскольку горутины выполняются одновременно, они могут влиять друг на друга, поэтому для более точных замеров время каждой реализации дополнительно выводится при отдельном запуске (пункты 3 и 4 меню).

```
func checkTime(g Graph, start int, f func(int, *[]int), wg *sync.WaitGroup,
tm *time.Duration, order1 *[]int) {
    defer wg.Done()
    startTime := time.Now()
    f(start, order1)
    *tm = time.Since(startTime)
}

func compareTime(g Graph, start int, writeResult bool) {
    fmt.Println("\nСравнение времени выполнения BFS:")

    var order1, order2 []int
    var time1, time2 time.Duration
    var wg sync.WaitGroup

    wg.Add(1)
    go checkTime(g, start, g.BFSMatrix, &wg, &time1, &order1)
    wg.Add(1)
    go checkTime(g, start, g.BFSLinkedQueue, &wg, &time2, &order2)

    wg.Wait()

    if writeResult {
        printOrder("BFS со слайсом:", order1)
        printOrder("BFS с самостоятельно реализованной очередью:", order2)
    }

    fmt.Printf("\nВремя BFS со слайсом: %v\n", time1)
    fmt.Printf("Время BFS с односвязным списком: %v\n", time2)
}
```

Ввод, вывод и меню

Функции PrintMatrix и PrintAdjacencyList выводят матрицу и списки смежности с нумерацией вершин с единицы, функция printOrder выводит порядок посещения в виде цепочки через «->». Функции readInt и readProbability читают число и проверяют его диапазон, возвращая ошибку при некорректном вводе. Главная функция main в цикле выводит меню, читает

выбранный пункт и вызывает соответствующий метод; для пунктов, выполняющих обход, дополнительно измеряется и выводится время выполнения.

```
func (g Graph) PrintMatrix() {
    if g.N == 0 {
        fmt.Println("Граф ещё не создан.")
        return
    }

    fmt.Println("\nМатрица смежности:")
    fmt.Print("    ")

    for j := 0; j < g.N; j++ {
        fmt.Printf("%3d", j+1)
    }
    fmt.Println()

    for i := 0; i < g.N; i++ {
        fmt.Printf("%3d ", i+1)

        for j := 0; j < g.N; j++ {
            fmt.Printf("%3d", g.Mat[i][j])
        }

        fmt.Println()
    }
}

func (g Graph) PrintAdjacencyList() {
    if g.N == 0 {
        fmt.Println("Граф ещё не создан.")
        return
    }

    fmt.Println("\nСписки смежности:")
    list := g.AdjacencyList()

    for i, neighbors := range list {
        fmt.Printf("%d: ", i+1)

        if len(neighbors) == 0 {
            fmt.Println("нет смежных вершин")
            continue
        }

        for k, v := range neighbors {
            if k > 0 {
                fmt.Print(", ")
            }
            fmt.Print(v + 1)
        }

        fmt.Println()
    }
}

func printOrder(title string, order []int) {
    fmt.Println("\n" + title)
    fmt.Print("Порядок посещения вершин: ")
```

```
    for i, v := range order {
        if i > 0 {
            fmt.Print(" -> ")
        }

        fmt.Print(v + 1)
    }

    fmt.Println()
}

func readInt(in *bufio.Reader, prompt string, min, max int) (int, error) {
    fmt.Print(prompt)

    var v int
    _, err := fmt.Fscan(in, &v)
    if err != nil {
        return 0, fmt.Errorf("Ошибка ввода: %v", err)
    }

    if v >= min && v <= max {
        return v, nil
    }

    return 0, fmt.Errorf("Ошибка диапазона")
}

func readProbability(in *bufio.Reader) (float64, error) {
    fmt.Print("Вероятность появления ребра (0-100%): ")

    var p float64
    _, err := fmt.Fscan(in, &p)
    if err != nil {
        return 0, fmt.Errorf("Ошибка ввода вероятности: %v", err)
    }

    if p >= 0 && p <= 100 {
        return p, nil
    }
    return 0, fmt.Errorf("Ошибка диапазона вероятности")
}

func main() {
    rand.Seed(time.Now().UnixNano())

    in := bufio.NewReader(os.Stdin)

    var g Graph
    var probability float64 = 30

    var order []int

    for {
        fmt.Println("\n========== ЛАБОРАТОРНАЯ РАБОТА №8 ==========")
        fmt.Println("Обход графа в ширину (BFS)")
        fmt.Println("1. Создать случайный граф")
        fmt.Println("2. Вывести матрицу смежности")
        fmt.Println("3. BFS по матрице смежности (слайс)")
        fmt.Println("4. BFS по матрице (самостоятельная очередь)")
        fmt.Println("5. Сравнить время двух реализаций BFS")
        fmt.Println("6. Вывести списки смежности")
        fmt.Println("7. BFS по спискам смежности")
        fmt.Println("8. Указать вероятность появления ребра")
        fmt.Println("9. BFS по спискам смежности (очередь)")
```

```
        fmt.Println("0. Выход")

        choice, err := readInt(in, "Выберите пункт меню: ", 0, 9)
        for err != nil {
            fmt.Println("Ошибка:", err)
            choice, err = readInt(in, "Выберите пункт меню: ", 0, 9)
        }

        switch choice {
        case 0:
            fmt.Println("Работа программы завершена.")
            return

        case 1:
            n, err := readInt(in, "Количество вершин (1-100): ", 1, 100000)
            for err != nil {
                fmt.Println("Ошибка:", err)
                n, err = readInt(in, "Количество вершин (1-100): ", 1,
100000)
            }

            g = NewGraph(n, probability)

            fmt.Printf(
                "Граф создан: вершин — %d, вероятность каждого ребра —
%.2f%%.\n",
                n,
                probability,
            )

            g.PrintMatrix()

        case 2:
            g.PrintMatrix()

        case 3:
            if g.N == 0 {
                fmt.Println("Сначала создайте граф (пункт 1).")
                continue
            }

            start, err := readInt(in, "Введите начальную вершину: ", 1, g.N)
            for err != nil {
                fmt.Println("Ошибка:", err)
                start, err = readInt(in, "Введите начальную вершину: ", 1,
g.N)
            }
            start--

            startTime := time.Now()
            g.BFSMatrix(start, &order)
            elapsed := time.Since(startTime)

            printOrder("Обход в ширину по матрице смежности:", order)
            fmt.Printf("Время выполнения: %v\n", elapsed)

        case 4:
            if g.N == 0 {
                fmt.Println("Сначала создайте граф (пункт 1).")
                continue
            }

            start, err := readInt(in, "Введите начальную вершину: ", 1, g.N)
            for err != nil {
```

```
                fmt.Println("Ошибка:", err)
                start, err = readInt(in, "Введите начальную вершину: ", 1,
g.N)
            }
            start--

            startTime := time.Now()
            g.BFSLinkedQueue(start, &order)
            elapsed := time.Since(startTime)

            printOrder("Обход в ширину с самостоятельно реализованной
очередью:", order)
            fmt.Printf("Время выполнения: %v\n", elapsed)

        case 5:
            if g.N == 0 {
                fmt.Println("Сначала создайте граф (пункт 1).")
                continue
            }

            start, err := readInt(in, "Введите начальную вершину: ", 1, g.N)
            for err != nil {
                fmt.Println("Ошибка:", err)
                start, err = readInt(in, "Введите начальную вершину: ", 1,
g.N)
            }

            compareTime(g, start, false)

        case 6:
            g.PrintAdjacencyList()

        case 7:
            if g.N == 0 {
                fmt.Println("Сначала создайте граф (пункт 1).")
                continue
            }

            start, err := readInt(in, "Введите начальную вершину: ", 1, g.N)
            for err != nil {
                fmt.Println("Ошибка:", err)
                start, err = readInt(in, "Введите начальную вершину: ", 1,
g.N)
            }
            start--

            startTime := time.Now()
            g.BFSAdjacencyList(start, &order)
            elapsed := time.Since(startTime)

            printOrder("Обход в ширину по спискам смежности:", order)
            fmt.Printf("Время выполнения: %v\n", elapsed)

        case 8:
            probability, err := readProbability(in)
            for err != nil {
                fmt.Println(err)
                probability, err = readProbability(in)
            }

            fmt.Printf(
                "Установлена вероятность %.2f%%. Она будет использована при
следующей генерации графа.\n",
                probability,
```

```
            )
        case 9:
            if g.N == 0 {
                fmt.Println("Сначала создайте граф (пункт 1).")
                continue
            }

            start, err := readInt(in, "Введите начальную вершину: ", 1, g.N)
            for err != nil {
                fmt.Println("Ошибка:", err)
                start, err = readInt(in, "Введите начальную вершину: ", 1,
g.N)
            }
            start--

            startTime := time.Now()
            order := g.BFSAdjacencyListQ(start)
            elapsed := time.Since(startTime)

            printOrder("Обход в ширину по спискам смежности:", order)
            fmt.Printf("Время выполнения: %v\n", elapsed)
        }
    }
}
```

Оценка сложности

Пусть n — число вершин графа, m — число рёбер. При обходе по матрице смежности каждая вершина помещается в очередь и извлекается из неё ровно один раз, а при извлечении просматривается вся её строка из n элементов, поэтому время работы составляет $O(n^2)$ независимо от числа рёбер. При использовании списков смежности при обработке вершины просматривается только список её соседей, а суммарная длина всех списков пропорциональна числу рёбер, поэтому сам обход выполняется за $O(n + m)$. Построение списков из матрицы в программе требует просмотра всей матрицы и занимает $O(n^2)$, поэтому для матричного ввода выигрыш получается только при многократных обходах. Обе реализации очереди выполняют добавление и извлечение за $O(1)$: в срезе добавление выполняется за амортизированное $O(1)$, а извлечение сводится к сдвигу начала среза, в списке — за счёт указателя Last. Поэтому асимптотическая сложность обоих вариантов обхода по матрице одинакова, а различие во времени определяется постоянным множителем: список выделяет память под каждый узел отдельно, а срез хранит элементы одним непрерывным блоком. Дополнительная память во всех

вариантах составляет O(n) под отметки, порядок обхода и очередь, в которой одновременно находится не более n вершин.

Результаты работы программы

Ниже приведены примеры обхода. Первый пример — граф из шести вершин с рёбрами 1–2, 1–3, 1–6, 2–4 и 3–5 (граф из методических указаний), начальная вершина 1.

```
Матрица смежности:
   1  2  3  4  5  6
1  0  1  1  0  0  1
2  1  0  0  1  0  0
3  1  0  0  0  1  0
4  0  1  0  0  0  0
5  0  0  1  0  0  0
6  1  0  0  0  0  0
```

Рисунок 1 — Сгенерированная матрица смежности

```
Списки смежности:
1: 2, 3, 6
2: 1, 4
3: 1, 5
4: 2
5: 3
6: 1
```

Рисунок 2 — Списки смежности графа

Обход начинается с вершины 1: она помещается в очередь и помечается. После извлечения вершины 1 в очередь добавляются её соседи, так что Q = {2, 3, 6}. Затем извлекается вершина 2, и в очередь добавляется вершина 4 (Q = {3, 6, 4}); из вершины 3 добавляется вершина 5 (Q = {6, 4, 5}). Вершина 6 новых соседей не даёт (Q = {4, 5}), после чего извлекаются вершины 4 и 5, и очередь становится пустой. Вершины посещаются по уровням удалённости: сначала вершины 2, 3 и 6 на расстоянии в одно ребро, затем вершины 4 и 5 на расстоянии в два ребра. Все варианты обхода дают один и тот же порядок.

```
Обход в ширину по матрице смежности:
Порядок посещения вершин: 1 -> 2 -> 3 -> 6 -> 4 -> 5

Обход в ширину с самостоятельно реализованной очередью:
Порядок посещения вершин: 1 -> 2 -> 3 -> 6 -> 4 -> 5

Обход в ширину по спискам смежности:
Порядок посещения вершин: 1 -> 2 -> 3 -> 6 -> 4 -> 5
```

Рисунок 3 — Результаты обхода графа из шести вершин

Второй пример — несвязный граф из семи вершин с рёбрами 1–2, 1–3, 2–3, 4–5 и 5–6; у вершины 3 есть петля, вершина 7 изолирована. Начальная вершина 2.

```
Матрица смежности:
   1  2  3  4  5  6  7
1  0  1  1  0  0  0  0
2  1  0  1  0  0  0  0
3  1  1  1  0  0  0  0
4  0  0  0  0  1  0  0
5  0  0  0  1  0  1  0
6  0  0  0  0  1  0  0
7  0  0  0  0  0  0  0
```

Рисунок 4 — Матрица смежности несвязного графа

```
Списки смежности:
1: 2, 3
2: 1, 3
3: 1, 2, 3
4: 5
5: 4, 6
6: 5
7: нет смежных вершин
```

Рисунок 5 — Списки смежности несвязного графа

Обход из вершины 2 проходит первую компоненту связности (2, 1, 3); петля у вершины 3 не добавляет ничего в очередь, так как вершина 3 уже помечена. После этого алгоритм находит непосещённую вершину 4 и обходит вторую компоненту (4, 5, 6), а затем отдельно посещает изолированную вершину 7.

```
Обход в ширину по матрице смежности:
Порядок посещения вершин: 2 -> 1 -> 3 -> 4 -> 5 -> 6 -> 7
```

Рисунок 6 — Обход несвязного графа с начальной вершиной 2

Вывод

В ходе выполнения лабораторной работы был изучен алгоритм обхода графа в ширину, и на языке Go реализованы его варианты для матрицы смежности (с очередью на основе среза и с самостоятельно реализованной очередью на основе односвязного списка) и для списков смежности. Все реализации дают одинаковый порядок посещения вершин — по уровням удалённости от начальной вершины — и корректно обходят несвязные графы.

Анализ показал, что обход по матрице смежности имеет сложность $O(n^2)$, а по спискам смежности — $O(n + m)$, при этом обе реализации очереди выполняют операции за $O(1)$ и различаются лишь постоянным множителем. Результаты работы программы совпали с ожидаемыми, что подтверждает корректность реализации.